

Digital Skills Gaps and Learning Barriers in Technology Education: An Integrative Review in Multilingual Contexts

Rodrigo Moreira dos Santos

Jala University

rodrigo.moreira@jala.university

Marcos Vinicius Bueno de Moraes

Jala University

marcos.bueno@jala.university

Sandi Delevante

Jala University

sandi.delevante@jala.university

BIOGRAPHIES

Rodrigo Moreira dos Santos is a professor, researcher, and technology professional affiliated with Jala University. His academic background includes Information Systems and Internet Technologies, Software Engineering, Big Data and Business Intelligence, Blockchain, and Higher Education Teaching. He has experience in higher and professional education, teaching programming, databases, software development, cloud computing, cybersecurity, artificial intelligence, and digital technologies. His research interests include technology education, digital skills, artificial intelligence, Internet of Things, software engineering, and data analysis. His academic and professional work integrates teaching, research, curriculum development, and practical technology applications, with emphasis on digital inclusion and computational competencies in educational contexts.

Marcos Vinicius Bueno de Moraes is a professor, researcher, and technology professional affiliated with Jala University. His interdisciplinary academic background spans Mathematics, Physics, Computer Science, Software Engineering, Artificial Intelligence, and data analysis. He has extensive experience in higher education, teaching Logic, Discrete Mathematics, Calculus, programming, and modelling, while also contributing to academic management, curriculum development, faculty coordination, and educational quality initiatives. His research interests include atmospheric modeling, meteorological data analysis, mathematical modeling, Artificial Intelligence, and deep learning. His professional work integrates teaching, research, technology, academic leadership, and interdisciplinary education, with a focus on practical solutions to real-world problems and contexts.

Sandi is an academic leader and researcher whose work focuses on digital literacy, artificial intelligence, and technology in higher education. Her interests include digital inclusion, multilingual access, and the development of digital skills among students and faculty. She has experience in academic management, faculty development, curriculum quality, and higher education policy. Her research examines how institutions can better prepare learners for changing academic and professional environments. She is particularly interested in practical approaches that strengthen digital capacity and responsible use of emerging technologies.

STRUCTURED ABSTRACT

[Purpose] The expansion of technology education has increased students' access to programming, computational thinking, and different forms of digital production. However, differences in prior knowledge, digital competence, problem-solving ability, linguistic background, and institutional conditions may create unequal learning conditions. This study analyzes the main skills gaps and learning barriers associated with technology education, with particular attention to programming education and multilingual contexts, and examines their implications for digital inclusion, digital-skills policy, AI governance, institutional capacity, and equitable access to computational knowledge in Latin America.

[Methodology/approach/design] A structured integrative literature review was conducted, incorporating transparency principles from PRISMA 2020 (Page et al., 2021). The review is not presented as an exhaustive systematic review. Instead, it identifies recurring themes and conceptual relationships across a multidisciplinary corpus published mainly between 2018 and August 2026, complemented by earlier conceptual references. The literature was organized around five research questions and analyzed through thematic synthesis covering digital competence, cognitive factors, linguistic barriers, pedagogical strategies, and the Latin American context.

[Findings] The synthesis indicates that learning difficulties in technology education are multidimensional. Prior knowledge, abstraction, decomposition, problem solving, and self-efficacy form the cognitive dimension. Autonomy, information search and evaluation, digital problem solving, and technological production characterize the digital dimension. Language, interpretation of task statements, computing terminology, documentation, and communication about code form the linguistic dimension. The literature also indicates that diagnosis, scaffolding, feedback, visualization, debugging, and collaborative learning may operate as educational mediators.

[Practical implications] Educators and institutions may use initial diagnosis, progressive support, explicit teaching of technical terminology, debugging activities, and assessments centered on understanding and problem solving. At policy level, the findings support combining infrastructure and connectivity with digital-skills development, multilingual access to technical resources, institutional capacity building, and clear AI-governance guidance for the

educational use of generative artificial intelligence. These priorities also affect access to information, communication equity, and participation in digital society.

[Originality/value] The study integrates research areas that are often examined separately and proposes a conceptual model in which digital, cognitive, and linguistic conditions interact with educational mediation. The contribution broadens the understanding of digital inclusion by distinguishing access and everyday technology use from autonomy and the capacity for technological production.

Keywords: Digital Skills. Learning Barriers. Technology Education. Multilingual Contexts. Programming Education.

INTRODUCTION

The increasing digitalization of society has expanded the importance of the knowledge and competencies required to use, understand, and produce digital technologies. Computational systems are now present across professional and social domains, changing communication, production, learning, and decision-making processes. As a result, schools, technical education institutions, and universities have increasingly incorporated content related to programming, computational thinking, data analysis, software development, and the critical use of digital technologies.

However, everyday exposure to digital devices and applications does not necessarily mean that students possess the competencies required to use technology autonomously and productively. There is an important distinction between using ready-made applications and understanding problems, selecting tools, evaluating information, and building technological solutions.

This distinction is supported by the Digital Competence Framework for Citizens (DigComp. Version 2.2); (Vuorikari, Kluzer and Punie, 2022), which organizes digital competence into five broad areas: information and data literacy, communication and collaboration, digital content creation, safety, and problem solving. The framework demonstrates that digital competence extends beyond the operational use of devices and includes capabilities related to understanding, evaluation, adaptation, and production.

The recent evolution of DigComp further reinforces this perspective. Cosgrove and Cachia (2025) shows that DigComp 3.0 updates the framework in response to recent technological changes and integrates artificial intelligence competence across its structure, while also strengthening issues such as cybersecurity, disinformation, digital rights, and digital well-being. Digital competence should therefore be understood as a dynamic combination of knowledge, skills, and attitudes that evolves as technological environments change.

This broader understanding has important educational implications. The presence of computers, digital platforms, or artificial intelligence systems in educational environments does not, by itself, guarantee positive learning outcomes. The UNESCO Global Education Monitoring Report 2023 (UNESCO, 2023a) emphasizes that the educational effects of technology depend on factors such as access, equity, content quality, teacher preparation, governance, and institutional conditions.

Similarly, the OECD (2023) describes digital education as an ecosystem composed of infrastructure, tools, learning resources, information systems, assessment, human competencies, and institutional capacity. This perspective shifts the discussion away from the simple availability of technology toward the conditions that enable its meaningful use.

The issue becomes particularly relevant when the focus is programming education.

Programming requires more than memorizing commands or reproducing syntactic structures. Before writing code, students must understand the problem, identify relevant information, establish relationships, decompose the situation into smaller parts, formulate a solution strategy, and represent that strategy precisely enough to be transformed into an algorithm.

These capabilities are closely related to computational thinking. Wing (2006) played an important role in popularizing this concept by describing it as a form of problem solving grounded in fundamental ideas from Computer Science and applicable beyond software development.

Subsequent research broadened the discussion and demonstrated that computational thinking is a multidimensional construct. Tang, Yin, Lin, Hadad and Zhai (2020), in a systematic review of computational thinking assessment, identified considerable diversity in definitions, instruments, and measurement practices. Tikva and Tambouris (2021), reviewing 101 studies on computational thinking through programming, identified six major research areas: knowledge base, learning strategies, assessment, tools, learner-related factors, and capacity building. These findings indicate that learning programming does not depend on a single ability. Introductory programming courses, often referred to as CS1 in the Computing Education literature, are also characterized by considerable student heterogeneity. A single class may include students with prior programming experience, students with strong

familiarity with computers, and students encountering concepts such as variables, conditions, loops, functions, or data structures for the first time.

Prior knowledge can take several forms. Some students may already know a programming language. Others may never have written code but may possess substantial experience in structured problem solving. Some may be comfortable with digital devices but unfamiliar with development environments, file systems, terminals, or technical documentation.

Digital experience should therefore not be treated as a binary variable.

Research on introductory programming has identified relationships between learning outcomes and factors such as prior knowledge, self-efficacy, and specific problem-solving abilities. These associations, however, should not be interpreted deterministically. Lack of previous programming experience does not imply an inability to learn, just as previous experience does not necessarily imply deep conceptual understanding. This distinction has important pedagogical consequences.

When initial differences are ignored, a curriculum may implicitly favor students whose prior experience is already close to the expected starting point. These students may understand the first concepts more quickly, complete more activities, and accumulate additional experience, while other students must simultaneously learn computational concepts, unfamiliar tools, and new forms of representation.

In this context, diagnosis and scaffolding may play an important role in reducing initial learning gaps. The linguistic dimension adds another layer to technology education. Computer Science has a strong relationship with the English language. Programming keywords, error messages, libraries, documentation, forums, repositories, and a substantial portion of professional terminology are predominantly presented in English. Consequently, a Brazilian or other Latin American student may participate in a course taught in their first language while still being immersed in a functionally multilingual learning environment.

A lecture may be delivered in Portuguese. The task statement may be written in Portuguese. The programming language uses English-derived terminology. Documentation may be written in English. Error messages may be displayed in English. Resources found in international technical forums are frequently written in English. Learning therefore occurs through the interaction of natural language, technical language, and formal language.

Research in Computing Education has examined this relationship directly. Soosai Raj, Ketsuriyong, Patel and Halverson (2018) investigated the role of native language in programming education. Guo (2018) identified barriers faced by non-native English speakers while learning programming. Later studies examined computing terminology, the experiences of non-native English-speaking students in introductory courses, and multilingual approaches to Computer Science education.

Generative artificial intelligence introduces a new dimension to this linguistic challenge. Large language models allow students to request code explanations, translate documentation, and interpret technical messages using their preferred language. Prather et al. (2025), for example, investigated multilingual prompting in programming education using Arabic, Chinese, and Portuguese. This capability may reduce some linguistic barriers, but it also creates another educational problem. Receiving a correct answer does not necessarily imply understanding the solution. A student may use artificial intelligence to generate a functional program while lacking the knowledge required to evaluate its logic, identify vulnerabilities, recognize edge cases, or modify the code when requirements change.

The expansion of generative tools therefore does not reduce the importance of digital competence. In some situations, it increases the importance of critical judgment. This transformation also affects traditional forms of assessment. When automated systems can generate code from natural-language descriptions, evaluating only the final program provides increasingly limited information about what a student actually understands. These issues are particularly relevant in Latin America. The region has expanded policies and initiatives related to digital education, computational thinking, coding, and Computer Science. Studies prepared for the Global Education Monitoring Report examine both the development of programming and coding skills and the inclusion of Computer Science as a curriculum subject in Latin American countries (UNESCO, 2023b).

However, this expansion is not homogeneous. Educational systems differ in infrastructure, connectivity, teacher preparation, economic development, and access to technology education. In addition, Portuguese and Spanish dominate formal education across much of the region, while the international technology ecosystem remains largely structured around English. This combination creates learning environments in which digital inequalities, differences in academic preparation, and linguistic barriers may coexist. Existing research often examines these dimensions separately. Research on digital inequality examines access, use, and the benefits derived from technology. Computing Education research investigates computational thinking, programming learning, misconceptions, debugging, and pedagogical strategies. Research on multilingual education examines the role of language in access

to knowledge. Integrating these perspectives provides an opportunity to understand learning barriers in technology education in a less reduced way.

From a policy perspective, these differences extend beyond classroom performance. Unequal infrastructure and connectivity can restrict access to digital information and learning opportunities; linguistic barriers can limit participation in technical communities and access to professional knowledge; and uneven institutional capacity can determine whether schools and universities are able to implement digital-skills and AI policies effectively. Technology education should therefore be viewed not only as a curricular issue, but also as part of a broader regional agenda involving digital inclusion, communication equity, multilingual access, AI governance, and institutional capacity (ECLAC, 2022; OECD, 2023, 2026; UNESCO, 2023a, 2023b).

Against this background, the objective of this study is to analyze, through an integrative literature review, the main digital skills gaps and learning barriers associated with technology education, with particular attention to programming education and multilingual contexts.

The study is guided by the following research questions:

- RQ1. Which gaps in prior knowledge and prior competencies are most frequently associated with difficulties in learning programming and technology-related content?
- RQ2. How can linguistic barriers and the predominance of English in Computing affect learning in multilingual contexts?
- RQ3. Which dimensions of digital competence are associated in the literature with the ability to solve problems and produce technological solutions?
- RQ4. Which pedagogical strategies are identified as appropriate for reducing cognitive, digital, and linguistic learning barriers?
- RQ5. Which aspects of this discussion are particularly relevant to technology education in Latin America?

The main contribution of this paper is the integration of these different lines of research into an Integrated Model of Barriers and Mediation in Technology Education. The model organizes digital, cognitive, and linguistic conditions and positions the educational dimension as a mediating space between these conditions and the progressive development of autonomy and technological production.

THEORETICAL BACKGROUND

Digital Competence and Digital Inequality

The concept of digital competence has evolved alongside changes in how people interact with technology. During the early expansion of personal computing and internet access, discussions of digital inequality focused heavily on whether individuals had access to devices and connectivity. As access expanded, however, it became increasingly clear that infrastructure represented only one dimension of digital inequality. Van Dijk (2020) argues that digital inequality should be understood as a process involving different forms of access and appropriation. Motivation, material access, skills, patterns of use, and the outcomes obtained through technology may create substantial differences even among individuals who appear to have similar levels of access. This distinction is particularly relevant in technology education. Two students may use the same computer laboratory, access the same learning platform, and receive the same instructional materials while still displaying markedly different levels of technological autonomy.

One student may already be familiar with file systems, integrated development environments, software installation, technical documentation, and troubleshooting. Another may use digital technologies intensively in everyday life but have experience concentrated mainly on social media, video platforms, messaging applications, and mobile interfaces. Both students have digital experience, but they bring different technological repertoires to the learning environment. This distinction challenges simplified assumptions that frequent use of digital devices automatically produces advanced digital competence. The DigComp, provides a useful conceptual structure for understanding this difference. The 2.2 version organizes digital competence into five major areas: information and data literacy, communication and collaboration, digital content creation, safety, and problem solving (Vuorikari, Kluzer and Punie, 2022). This structure shows that digital competence extends beyond tool operation. Finding information is different from evaluating its reliability. Consuming content is different from producing it. Using an application under normal conditions is different from diagnosing a problem when the technology behaves unexpectedly. Problem solving therefore occupies a central position in this broader understanding of digital competence.

DigComp 3.0 maintains the framework's five competence areas while updating learning outcomes in response to recent developments in the digital ecosystem. Artificial intelligence competence is integrated throughout the framework, together with greater attention to cybersecurity, misinformation, digital rights, and digital well-being (Cosgrove and Cachia, 2025). This evolution is important for technology education because it reinforces the idea

that digital competence is not a fixed endpoint. New technologies create new forms of interaction and also create new competencies required to use and evaluate them critically.

For the purposes of technology education, this study proposes an analytical progression consisting of five elements: Access → Use → Understanding → Problem Solving → Technological Production.

Figure 1 represents a conceptual synthesis developed in this study from the literature on digital competence and digital inequality. The model does not define mandatory levels or a strictly linear developmental sequence. Students may display high competence in one area and limited competence in another. The first element, access, refers to the availability of devices, connectivity, software, and other resources required for digital participation. The second, use, concerns the ability to interact with digital applications and services. The third, understanding, includes the ability to interpret information, concepts, operations, and system behavior. The fourth, problem solving, refers to identifying difficulties, selecting strategies, investigating causes, and adapting prior knowledge to unfamiliar situations. Finally, technological production concerns the ability to create digital artifacts, algorithms, programs, automations, or other computational solutions.

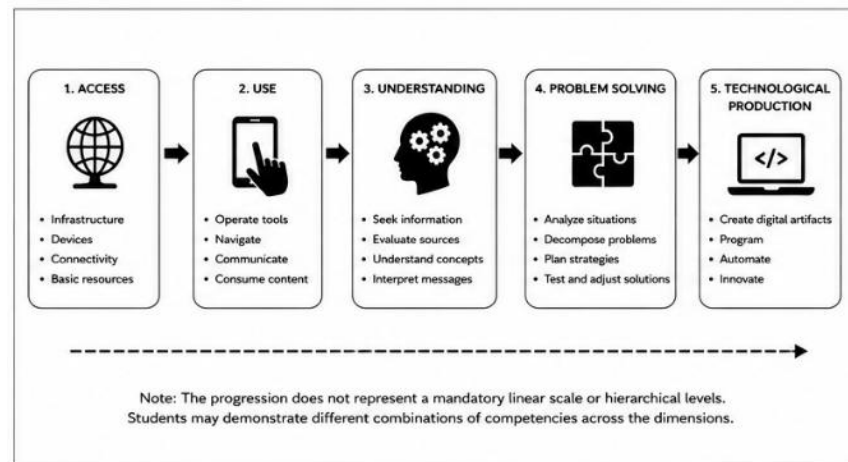


Figure 1. Analytical progression of technological participation in education.

This analytical organization helps explain why students may be highly familiar with everyday technology while still experiencing substantial difficulty in programming education. Using social media, editing short videos, or navigating mobile applications does not necessarily require students to decompose problems, create algorithmic representations, interpret exceptions, or evaluate technical documentation. Technology education should therefore avoid assuming that frequent exposure to digital environments automatically develops the competencies required for technological production. This distinction becomes even more important with the increasing availability of generative artificial intelligence. Systems capable of generating code from natural-language instructions reduce some operational barriers. At the same time, they make evaluative competencies more important. A student may obtain an apparently correct implementation while still needing to determine: whether the logic satisfies the requirements, whether important cases were omitted, whether the code introduces security risks, whether the results are reliable, and whether the solution can be adapted when requirements change. Generative artificial intelligence therefore does not necessarily eliminate competence gaps. In some contexts, it changes which competencies become most important.

Computational Thinking and Problem Solving

Programming education provides a particularly relevant context for examining the interaction between digital competence and cognitive processes. Although writing code requires knowledge of syntax and language semantics, developing a computational solution requires several stages that precede implementation. Students must understand the problem, identify relevant information, remove unnecessary details, decompose the situation, recognize patterns, formulate a strategy, and transform that strategy into a representation precise enough to be implemented. These capabilities are closely associated with computational thinking.

Wing (2006) contributed significantly to the popularization of computational thinking by describing it as a problem-solving approach grounded in fundamental ideas from Computer Science. Following this work, computational thinking gained increasing prominence in educational policy, curriculum design, and Computing Education research.

However, later studies show that computational thinking does not have a single operational definition. Tang, Yin, Lin, Hadad and Zhai (2020), in a systematic review of empirical studies on computational thinking assessment, identified considerable variation in instruments, constructs, and educational contexts. Their findings demonstrate that computational thinking should not be treated as an automatic synonym for intelligence, logical reasoning, or programming ability. Tikva and Tambouris (2021) reviewed 101 studies related to the development of computational thinking through programming in K-12 (primary and secondary school) education and proposed a model organized around six areas: Knowledge Base, Learning Strategies, Assessment, Tools, Factors, and Capacity Building. These findings support a multidimensional interpretation. Computational thinking involves knowledge and practices that interact with tools, learning strategies, assessment approaches, and learner characteristics. In technology education, some of these practices include: decomposition, abstraction, pattern recognition, algorithm design, testing, revision, and generalization. Programming activities provide repeated opportunities to mobilize these processes.

Prior Knowledge

Prior knowledge appears consistently in research on introductory programming education. However, it should not be treated as a single homogeneous variable. One student may enter a course with prior Python experience. Another may never have programmed but may have substantial experience with spreadsheets, formulas, and automation. Another may have strong mathematical preparation but no experience with development environments. Another may use computers frequently but have never interacted with a command line, compiler, or integrated development environment. These examples demonstrate that prior knowledge can take several forms. Therefore, asking only whether a student has programmed before provides an incomplete view of their starting point.

Pires et al. (2024) examined factors associated with readiness to learn programming in higher education and considered variables including prior knowledge, mathematical skills, and self-efficacy. Studies of this kind reinforce the importance of recognizing differences that exist before introductory courses begin. These associations should nevertheless be interpreted cautiously. Prior experience may provide an initial advantage, but it does not imply that students without such experience cannot later develop equivalent competencies. The pedagogical question is therefore how to reduce the impact of initial differences on learning opportunities.

Mathematics and Logical Reasoning

The relationship between mathematics and programming appears frequently in both academic research and educational discourse. The two domains share certain cognitive practices, including symbolic representation, abstraction, pattern recognition, and structured problem solving. However, this relationship should not be transformed into a deterministic assumption. Mathematical performance is not a complete measure of the ability to learn programming.

Students may experience difficulty with specific mathematical topics while developing strong algorithmic reasoning. Similarly, high mathematical performance does not guarantee immediate understanding of concepts related to program state, data structures, objects, or debugging.

A more appropriate interpretation is that some mathematical and logical repertoires may facilitate particular aspects of programming learning. This distinction has direct pedagogical implications. Consider the expression: `if age >= 18`. A student may understand the syntax while still lacking an adequate representation of comparison, Boolean expressions, or decision making. In this situation, repeating the syntax of `if` does not address the underlying conceptual difficulty. It may be necessary to work first with: `variable` → `value` → `comparison` → `condition` → `decision`. This decomposition helps separate the computational concept from its syntactic representation.

Mental Models and Misconceptions

Another important line of research in Computing Education focuses on misconceptions developed by novice programmers. A misconception occurs when a learner constructs an interpretation of program behavior that does not correspond to the actual behavior of the language or system. This is particularly important because incorrect code may still appear logically consistent from the student's perspective. For example, an error in a loop may result from an inaccurate mental model of when the condition is evaluated, how a variable changes, or when execution terminates. Simply showing the correct solution may not modify that mental model. Activities involving tracing, step-by-step execution, and output prediction can therefore provide greater pedagogical value. Students can be asked:

- What is the value of this variable before the first iteration?
- What happens after this statement executes?
- What will the value be during the next iteration?

- When does the condition become false?

These questions require students to make their internal model of execution explicit.

Self-Efficacy and Persistence

In addition to cognitive competencies, programming education research also examines students' beliefs about their own capabilities. Self-efficacy refers to an individual's belief in their ability to perform the actions required to achieve a particular goal (Bandura, 1997).

In programming, self-efficacy can influence persistence when errors occur. Programming activities generate frequent negative feedback. Programs fail to compile. Tests fail. Unexpected outputs appear. A correction may introduce another problem. For experienced programmers, these events are part of ordinary development practice. For beginners, repeated failure may be interpreted as evidence that they are incapable of programming. The way errors are framed pedagogically therefore matters. A learning environment that treats errors only as failure may contribute to reduced confidence. An environment that treats errors as evidence for investigation may support a different understanding of programming practice.

Debugging as a Competence

Debugging provides a clear example of this distinction. Debugging should not be understood only as correcting code. It involves:

1. observing a behavior;
2. comparing actual and expected results;
3. forming a hypothesis;
4. identifying possible causes;
5. executing tests;
6. revising the hypothesis;
7. modifying the solution;
8. testing again.

This process mobilizes both scientific and computational reasoning. Debugging can therefore be taught explicitly. Rather than assigning only tasks in which students build programs from scratch, educators may provide intentionally incorrect programs and ask students to investigate them. The objective changes from production to diagnosis. Such activities assess competencies that are not captured by syntax production alone.

The Cognitive Process of Programming

The integration of these elements allows programming problem solving to be represented as a process composed of several stages:

Problem Statement → Interpretation → Decomposition → Abstraction → Algorithm → Implementation → Execution → Debugging → Revision

Figure 2 emphasizes that implementation represents only one stage of the overall process. This observation leads to an important conclusion: An error observed in code may be only the final visible manifestation of a difficulty that occurred earlier in the problem-solving process. A student may produce incorrect code because they do not understand syntax. However, incorrect code may also result because the student: misunderstood the task statement, incorrectly identified inputs or outputs, failed to decompose the problem, selected an inappropriate strategy, designed an incorrect algorithm, and did not test the solution adequately. These situations require different forms of instructional intervention. Analysis of the final program alone therefore does not always provide sufficient information to diagnose the student's difficulty. This argument will be revisited in the Discussion section.

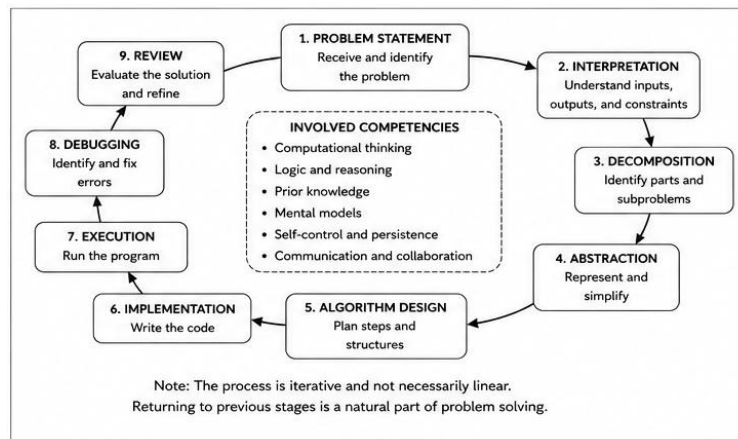


Figure 2. Cognitive process of problem solving in programming.

Language and Multilingualism in Computing Education

Language occupies a distinctive position in Computer Science education because students work simultaneously with natural language and formal language. In multilingual contexts, this relationship becomes more complex. A student may receive explanations in Portuguese or Spanish, write code using English-derived keywords, consult documentation in English, and receive error messages in English. Recent research indicates that this situation deserves specific attention. Guo (2018) investigated barriers faced by non-native English speakers learning programming through English-language resources and identified needs associated with the linguistic accessibility of educational materials. Soosai Raj, Ketsuriyongk, Patel and Halverson (2018) directly investigated whether native language plays a role in learning a programming language. Subsequent work by Alaofi examined computing terminology among students in introductory programming courses who do not speak English as a first language (Alaofi and Russell, 2021, 2022, 2023).

The existence of instruments specifically designed to assess computing terminology indicates that the problem extends beyond general language proficiency. There is a domain-specific layer of computational language. In this paper, this layer is referred to as technical-computing linguistic competence. The concept refers to the ability to understand and use specialized terminology required to participate effectively in computing activities. Examples include: instance, scope, runtime, dependency, exception, thread, request, response, endpoint, and payload. Many of these terms have everyday meanings in English while acquiring specialized meanings in Computing. General English proficiency and understanding of technical English should therefore not be treated as equivalent.

Five Linguistic Layers

The literature synthesis allows the linguistic dimension to be organized into five layers.

Language of Instruction

This refers to the language used by the instructor, institution, and learning materials. When the language of instruction is not the student's strongest language, learning may require simultaneous processing of linguistic and conceptual content.

Language of Task Statements

Students must understand the problem before they can program a solution. Terms that appear straightforward may create ambiguity. For example: "Traverse the array and determine the largest element located at an even position." A correct solution requires a precise understanding of: array, element, position, index, value, and parity.

A linguistic misunderstanding may appear to be a logical reasoning problem when only the final code is examined.

Programming Language

Keywords such as: if, while, return, class, and break are derived from English. However, translation alone does not necessarily solve the problem. Knowing that while can be translated as "enquanto" or "mientras" does not mean that the student understands how loop execution works.

Documentation and Tools

As students become more autonomous, their learning increasingly depends on access to external resources. Official documentation, libraries, issue trackers, forums, and error messages are frequently written in English. Consider the

message: TypeError: cannot read properties of undefined. To use this message as diagnostic information, the student must:

1. interpret it linguistically;
2. understand the technical concepts involved;
3. relate the concepts to the current program state;
4. formulate a hypothesis;
5. test a possible solution.

The difficulty therefore involves language, computational knowledge, and problem solving at the same time.

Communication About Code

Learning programming also means learning how to talk about programming. Students need to: formulate questions, explain solutions, discuss errors, justify design choices, interpret feedback, and collaborate. The ability to produce code and the ability to communicate computational reasoning are related but not identical. In multilingual contexts, the second may require explicit educational support.

Translanguaging

Translanguaging (García and Wei, 2014) provides a useful perspective for understanding this behavior. The concept describes situations in which learners use resources from different languages while constructing knowledge. In programming education, a student may: read the task in Portuguese; discuss the solution in Portuguese; write pseudocode in Portuguese; implement the solution in Python; consult English documentation; and later explain the algorithm using terminology from both languages. This alternation does not necessarily represent a deficiency. It may serve as a resource for gradually constructing understanding.

Artificial Intelligence as Linguistic Mediation

Generative artificial intelligence creates new possibilities in this context. Prather et al. (2025) investigated multilingual prompting in programming education, including interactions in Portuguese. A student may ask:

"Explain this error in Portuguese."

"Translate this documentation while preserving the technical terms."

"Explain what runtime means in this context."

"Do not solve the problem. Help me understand the task statement."

These capabilities may reduce some barriers to access. However, generative systems can also produce incorrect explanations or inappropriate simplifications. Linguistic accessibility must therefore remain associated with critical evaluation. Artificial intelligence may make an explanation easier to access linguistically without guaranteeing that the explanation is technically correct.

Latin American Contexts as Functionally Multilingual Environments

This analysis has particular relevance in Latin America. Portuguese and Spanish are the dominant instructional languages across much of the region. The global technology ecosystem, however, remains heavily structured around English. As a result, a programming course delivered entirely in Portuguese may still be functionally multilingual. The classroom language is Portuguese. The integrated development environment may be in English. The compiler may produce English messages. Documentation may be in English. Repositories may be written in English. Programming terminology is often English-based. This configuration justifies the analysis of multilingual contexts even when students are not enrolled in formally bilingual programs. It also demonstrates that linguistic competence is part of technological autonomy.

EDUCATIONAL DIMENSION AND MEDIATION OF LEARNING BARRIERS

The digital, cognitive, and linguistic dimensions described in the previous sections represent conditions that may influence the learning experience in technology education. However, these conditions do not determine student performance independently. The way teaching is designed may reduce, compensate for, or intensify specific difficulties. For this reason, the educational dimension occupies a distinct position in the conceptual model proposed in this study.

While the digital, cognitive, and linguistic dimensions describe repertoires and conditions related to students and their learning environments, the educational dimension represents the area in which educators and institutions have greater capacity for direct intervention. This mediation may occur through strategies such as diagnosis, scaffolding, feedback, visualization, active learning, collaboration, leveling activities, and differentiated instruction.

Diagnosis as a Pedagogical Tool

Student heterogeneity makes diagnostic activities particularly relevant at the beginning of introductory technology courses. The purpose of diagnosis should not be to classify students as capable or incapable of learning programming. The literature reviewed in this study does not support a deterministic interpretation in which specific prior competencies define who can or cannot develop computational skills.

Instead, diagnostic activities should identify support needs. Possible dimensions include: previous programming experience, familiarity with computers and digital tools, ability to interpret problem statements, basic logical reasoning, previous experience with structured problem solving, familiarity with computing terminology, ability to interpret technical documentation, and perceived self-efficacy. These results can guide different forms of support. Students with limited programming experience may receive additional materials on fundamental concepts. Students with more advanced experience may work on extension activities that preserve engagement without changing the learning objectives of the course. Differentiated instruction therefore does not necessarily require different curricula for every student. It may instead involve different levels of support around shared learning goals.

Scaffolding and Progressive Withdrawal of Support

Scaffolding refers to temporary support provided during learning so that students can perform tasks they are not yet able to complete independently, consistent with Vygotsky's (1978) concept of socially mediated learning. In programming education, scaffolding may take several forms: pseudocode, flowcharts, partially completed code, solved examples, hints, tracing tables, guiding questions, starter templates, and decomposition of larger problems. The central principle is that support should gradually decrease. If educators continuously provide the complete structure of a solution, students may complete tasks without developing equivalent autonomy. On the other hand, removing support before students have developed appropriate mental models may increase frustration and dependence on external assistance. Scaffolding can therefore be understood as a four-stage process: (Bolloju, Malapati and Upadhyay, 2026):

Diagnose → Support → Reduce → Transfer

During diagnosis, learning needs are identified. During support, intermediate representations and guidance are provided. In the next stage, those structures are gradually removed. Finally, students encounter new problems in which they must select and apply strategies independently. This approach frames support and autonomy as complementary rather than opposing goals.

Visualization of Computational Processes

Programming requires students to understand processes that are usually invisible. A student cannot physically observe the changing value of a variable, the creation of a function call, or the state changes within a data structure. Experienced programmers develop mental models capable of representing these processes internally. Beginners may benefit from external representations. Tracing tables, diagrams, memory models, state representations, and step-by-step execution can make computational processes more explicit. For example, the behavior of a loop can be represented in a table that shows the value of each variable before and after each iteration.

A conditional structure can be represented using a flowchart. A function can be represented in terms of inputs, processing, and outputs. These representations do not replace code. They act as bridges between abstract computational behavior and student understanding. As students become more experienced, this support can be gradually reduced.

Feedback and Debugging

Feedback is a constant feature of programming. Compilers provide feedback. Interpreters provide feedback. Tests provide feedback. Educators provide feedback. Peers provide feedback. Artificial intelligence systems may also provide feedback. However, not all feedback has the same pedagogical value. Simply informing students that an answer is incorrect may identify a problem without helping them reconstruct their reasoning. More effective feedback can guide investigation. Rocha, Costa and Tedesco (2023) show as an example: "Your loop is incorrect.", which offers limited support. A question such as: "Observe the value of the control variable after each iteration. At what point should the condition become false?" directs attention toward the reasoning process without immediately supplying the solution. This form of feedback keeps the learner involved in diagnosis. Debugging can also be taught explicitly. Students can be given intentionally incorrect programs and asked to:

1. reproduce the behavior;
2. identify symptoms;
3. formulate hypotheses;
4. locate possible causes;
5. modify the program;

6. run new tests;
7. explain why the correction works.

This process transforms errors from evidence of failure into evidence that can support investigation.

Active Learning

Programming is an inherently practical activity. Students may understand an explanation of conditional structures without being able to construct an appropriate condition in a new problem. For this reason, exposition should be combined with frequent opportunities for practice. A short instructional cycle may follow the sequence: Explanation → Example → Practice → Discussion → New Problem

This structure allows difficulties to become visible before they accumulate. It also provides opportunities for students to compare strategies and explain their reasoning.

Pair Programming and Collaborative Learning

Pair programming offers a form of collaborative learning that is particularly compatible with programming activities (Sheard, Malmi, Szabo, Kinnunen, Tshukudu, Craig and Mann, 2025). In a traditional configuration, one student acts as the driver, directly manipulating the code, while the other acts as the navigator, following the strategy, questioning decisions, and identifying possible problems. The roles should be exchanged regularly. This rotation matters because collaboration does not emerge automatically. When students have very different levels of experience, the more experienced participant may take control of the entire solution while the other becomes a passive observer. Pair programming therefore requires pedagogical structure. Its potential educational value lies partly in verbalization. When students explain a condition, justify a variable, or discuss an error message, they must make part of their reasoning explicit. This process also connects pair programming to the linguistic dimension. Students learn not only how to produce code but also how to communicate about code.

Generative Artificial Intelligence as Scaffolding

Generative artificial intelligence introduces new forms of support in technology education. Students may use such systems to request: explanations of error messages, translations of technical documentation, additional examples, test cases, comparisons between solutions, explanations of code, simplified technical descriptions, and guiding questions.

However, there is an important distinction between using artificial intelligence as scaffolding and using it as a substitute for intellectual activity. Consider the difference between: "Solve this problem." and: "Ask me three questions that can help me develop a strategy without giving me the solution."

The second interaction preserves a greater degree of student reasoning. Similarly: "Correct my code." differs from: "Explain what this error message means and indicate what I should inspect in my program." This distinction is especially important in programming education because automated systems can easily generate complete solutions. The educational question should therefore focus less on whether artificial intelligence is used and more on what cognitive activity remains under student responsibility.

Synthesis of the Educational Dimension

The literature reviewed in this study supports the organization of the educational dimension around seven main components: diagnosis, scaffolding, visualization, feedback, debugging, collaboration, and progressive development of autonomy.

Unlike the other dimensions, the educational dimension may function as a mediating mechanism. A cognitive difficulty may be addressed through visualization. A linguistic barrier may be reduced through glossaries and clearer instructional language. A digital difficulty may be addressed through guided use of tools. A difference in prior knowledge may be addressed through leveling activities. The educational dimension therefore occupies a central position in the integrated model presented later in the paper.

METHODOLOGY

Research Design

This study adopts a qualitative approach based on a structured integrative literature review. The method was selected because the research problem spans several areas, including: Computing Education, Educational Technology, Digital Competence, Computational Thinking, Multilingual Education, Programming Education, and Digital Inclusion.

An integrative review allows evidence from different methodological traditions to be combined into a broader conceptual synthesis. The review incorporated transparency principles associated with PRISMA 2020 (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) to organize and report the identification, screening, and selection of relevant publications. However, this study is not presented as a systematic review of intervention effectiveness, as a meta-analysis, or as an exhaustive mapping of all eligible evidence. PRISMA 2020 is used as a methodological reference for transparent reporting rather than as a claim that all procedures of a systematic

effectiveness review were applied. Accordingly, the findings are interpreted as recurring themes and conceptual relationships across the reviewed corpus, not as pooled effects or comprehensive coverage of all available evidence.

Research Questions

The review was guided by five research questions:

- RQ1. Which gaps in prior knowledge and prior competencies are most frequently associated with difficulties in learning programming and technology-related content?
- RQ2. How can linguistic barriers and the predominance of English in Computing affect learning in multilingual contexts?
- RQ3. Which dimensions of digital competence are associated in the literature with the ability to solve problems and produce technological solutions?
- RQ4. Which pedagogical strategies are identified as appropriate for reducing cognitive, digital, and linguistic learning barriers?
- RQ5. Which aspects of this discussion are particularly relevant to technology education in Latin America?

Information Sources

The search strategy considered academic databases and institutional sources relevant to the research areas examined. The main academic sources included: ACM Digital Library, IEEE Xplore, ScienceDirect, SpringerLink, ERIC, Scopus, and Web of Science, when records and corresponding publications were available for verification.

Institutional publications were also considered when they were relevant to the conceptual or regional context, particularly publications from: UNESCO, OECD, European Commission Joint Research Centre, and the Economic Commission for Latin America and the Caribbean, ECLAC. Institutional reports were used mainly to support the discussion of digital competence, education policy, digital inequality, and the Latin American context. Peer-reviewed research was prioritized for claims related directly to learning, student performance, and pedagogical intervention.

Publication Period

The main review corpus covers studies published between 2018 and August 2026. This period was selected to capture recent developments in: Computing Education, digital education, multilingual learning, generative artificial intelligence, and technology-related learning environments. Earlier publications were included when they were considered conceptually foundational. Examples include Wing (2006), Vygotsky (1978), and other established references. These older studies were used for conceptual grounding rather than as contemporary evidence of current educational conditions.

Languages

The review considered publications written in: English, Portuguese, and Spanish. The inclusion of Portuguese and Spanish reflects the Latin American scope of the study. This decision also attempts to reduce the risk of constructing the review exclusively from English-language literature.

Search Strategies

Search strategies were organized around the major conceptual themes of the study.

Digital Competence

("digital competence" OR "digital skills" OR "digital literacy" OR "digital divide") AND ("technology education" OR "computer science education" OR programming OR "higher education")

Prior Knowledge and Programming

(programming OR "introductory programming" OR CS1) AND ("prior knowledge" OR mathematics OR "logical reasoning" OR "problem solving" OR "self-efficacy")

Computational Thinking

("computational thinking" OR "algorithmic thinking") AND (programming OR coding) AND (education OR learning)

Multilingualism

(programming OR "computer science education") AND (multilingual OR bilingual OR "non-native English" OR "native language" OR translanguaging)

Linguistic Barriers

(programming OR coding OR "computer science") AND ("language barrier" OR "linguistic barrier" OR terminology OR vocabulary) AND (education OR student)

Pedagogical Strategies

("programming education" OR "introductory programming" OR CS1) AND (scaffolding OR feedback OR "pair programming" OR visualization OR "active learning")

Latin America

("computer science education" OR "technology education" OR "digital skills" OR programming) AND ("Latin America" OR Brazil OR Portuguese OR Spanish) The syntax of these strings was adapted when necessary to the requirements of specific search systems while preserving the same core concepts.

Inclusion Criteria

A publication was considered relevant when it met at least one of the following thematic criteria: investigated the teaching or learning of programming, examined digital competence in educational contexts, addressed computational thinking in relation to programming, investigated prior knowledge, problem solving, or self-efficacy, addressed multilingual or non-native English-speaking students, analyzed pedagogical strategies for novice programmers, and provided relevant evidence on digital education or Computing Education in Latin America.

The following documentary criteria were also applied: publication information had to be verifiable, the publication had to relate directly to at least one research question, the text had to be available in English, Portuguese, or Spanish, authorship and publication source had to be identifiable, and the source had to be academic or produced by a recognized institution.

Exclusion Criteria

Publications were excluded when they: had no direct relationship with education, focused exclusively on professional software development, used programming only as a peripheral topic, lacked sufficient methodological or bibliographic information, duplicated another included publication, consisted primarily of unsupported opinion, and could not be reliably verified.

When multiple versions of the same study were identified, the most complete published version was prioritized.

Study Selection Process

The selection process was organized into four main stages.

Identification

Potentially relevant publications were identified through searches in the selected databases and institutional sources. Backward snowballing was also used for particularly relevant publications to identify earlier studies cited by those works. Forward citation tracking was used when appropriate to identify later studies that extended or discussed influential publications.

Duplicate Removal

Duplicate records retrieved from multiple sources were consolidated.

Screening

Titles and abstracts were examined according to the inclusion and exclusion criteria. Records clearly outside the scope of the review were removed.

Eligibility

The remaining publications were examined in greater detail to determine their relevance to the research questions and their suitability for thematic synthesis.

Presents the qualitative workflow used to identify, screen, and assess the literature. Because this study is an integrative review and a complete auditable search log was not maintained throughout the process, a numerical PRISMA flow diagram is not reported. This limitation is stated explicitly, and the article does not claim exhaustive systematic coverage of all eligible publications.

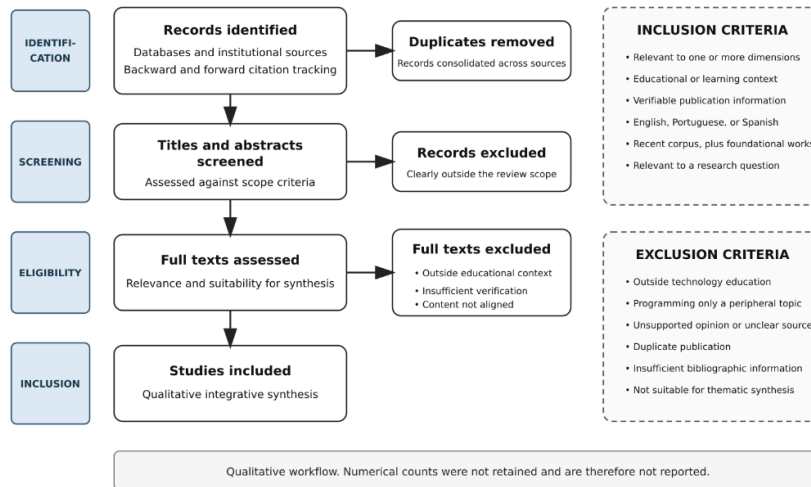


Figure 3. Process of identification and selection of the literature.

Data Extraction

For each included publication, the following information was extracted when available: author or authors, publication year, country or geographical context, educational level, participants or corpus, methodological design, technological or educational topic, main barrier or variable investigated, principal findings, limitations, related research question, and corresponding conceptual dimension. The extraction structure summarized in Table 1 was used to improve comparability across studies.

Field	Extracted information
Reference	Author and year
Context	Country, region, or population
Educational level	K-12, technical, or higher education
Participants	Population or corpus
Method	Quantitative, qualitative, mixed, or review
Topic	Main phenomenon investigated
Findings	Main contribution to the review
Dimension	Digital, cognitive, linguistic, or educational
RQ	Related research question

Table 1. Data extraction framework

Thematic Synthesis

The included literature was grouped according to thematic similarity. Four main analytical dimensions emerged from the synthesis.

Digital Dimension

This dimension included studies addressing: digital competence, digital autonomy, digital inequality, information search and evaluation, digital problem solving, and technological production.

Cognitive Dimension

This dimension included research on: prior knowledge, computational thinking, abstraction, decomposition, logical reasoning, mathematical background, self-efficacy, misconceptions, and debugging.

Linguistic Dimension

This dimension included studies addressing: language of instruction, non-native English-speaking students, computing terminology, task interpretation, technical documentation, communication about code, and multilingual learning environments.

Educational Dimension

This dimension included research on: scaffolding, feedback, active learning, pair programming, visualization, leveling activities, differentiated instruction, and the pedagogical use of artificial intelligence. Studies addressing more than one dimension were classified across all relevant categories.

Construction of the Conceptual Model

Following thematic synthesis, the identified dimensions were integrated into a conceptual model of learning barriers in technology education. The purpose of the model is not to assign statistical weights or establish causal relationships.

Instead, it organizes recurring relationships described in the literature and highlights how similar learning difficulties may emerge from different combinations of conditions. The initial synthesis considered four dimensions: digital, cognitive, linguistic, and educational. The analysis later refined the model by treating the educational dimension primarily as a mediating space between learner-related conditions and the development of autonomy and technological production. This refined structure is presented and discussed in the Discussion section.

Methodological Limitations

The methodological design also requires several limitations to be recognized. First, the literature examined spans different academic fields, each using its own terminology, theoretical frameworks, and measurement approaches. Second, concepts such as computational thinking and digital competence do not have a single universal operationalization. Third, the corpus includes studies conducted at different educational levels. Findings from K-12 settings cannot automatically be generalized to higher or technical education. Fourth, although Portuguese and Spanish were included, the international literature on Computing Education remains heavily concentrated in English. Finally, the present review is integrative and conceptual. Its findings identify patterns and relationships across the literature, but they do not support quantitative estimates of the causal impact of each barrier on student performance.

RESULTS

The literature reviewed in this study indicates that learning barriers in technology education are distributed across several dimensions rather than being explained by a single factor. The thematic synthesis showed recurring relationships involving prior knowledge, digital competence, language, problem solving, self-efficacy, and educational support. The results are presented according to the five research questions defined in the methodology.

RQ1. Prior Knowledge and Competence Gaps

The literature indicates that students begin introductory programming courses with markedly different repertoires. Prior knowledge is one of the most frequently discussed factors, but it is not represented consistently across studies. In some cases, prior knowledge refers specifically to previous programming experience.

In other studies, it includes mathematical background, familiarity with digital tools, logical reasoning, problem-solving experience, confidence with technology, or prior exposure to computational concepts. This variation is important because two students with no formal programming experience may still begin a course with very different levels of preparation.

A student may have extensive experience with spreadsheets, automation, technical troubleshooting, or digital creation. Another may use digital technologies frequently but primarily for communication, entertainment, and content consumption. The literature therefore supports a multidimensional interpretation of prior knowledge. Several recurring elements were identified: previous programming experience, computational thinking, abstraction, decomposition, problem-solving strategies, mathematical and logical reasoning in some contexts, self-efficacy, and mental models of program execution. Prior programming experience may provide an initial advantage because students are already familiar with concepts such as variables, control structures, or functions.

However, previous experience does not automatically guarantee deep understanding. Students may reproduce programming structures without understanding their underlying behavior. Conversely, students without previous programming experience may develop strong computational reasoning when appropriate instructional support is available. This distinction is particularly relevant when interpreting mathematics and logical reasoning.

Some studies include mathematical skills among variables associated with introductory programming performance. However, the literature does not support the conclusion that mathematical performance determines programming ability. Programming and mathematics share practices such as symbolic representation, abstraction, and structured problem solving, but programming also depends on competencies that are not captured by mathematics alone.

These include tracing program state, debugging, tool use, interpretation of error messages, and understanding execution over time. Self-efficacy also appears as an important factor. Students who believe they can successfully

engage with programming tasks may show greater persistence when errors occur. This matters because error, revision, and repeated testing are normal features of programming. Students who interpret errors as evidence of personal inability may disengage more quickly than students who interpret them as information for problem solving.

The literature also highlights misconceptions and inaccurate mental models. Novice programmers may construct incorrect assumptions about how loops, variables, functions, or data structures behave. When these misconceptions remain unaddressed, students may reproduce similar errors across different tasks. The results therefore suggest that difficulties observed in introductory programming may originate before code is written.

RQ1 Synthesis

The most frequently identified learner-related factors were: prior knowledge, problem-solving ability, abstraction, decomposition, computational thinking, self-efficacy, and mental models. These factors are associated with programming learning but should not be interpreted as independent or deterministic predictors of success.

RQ2. Linguistic Barriers in Multilingual Contexts

The literature indicates that language may affect programming learning at several levels. The first level concerns the language of instruction. When a course is taught in a language that is not the student's strongest language, conceptual learning may require simultaneous linguistic processing.

The second level concerns task interpretation. Programming exercises are often presented through written problem statements. Students must identify requirements, inputs, outputs, restrictions, and expected behavior before developing a solution. A misunderstanding at this stage may later appear as a programming error. The third level is technical terminology. Research involving non-native English-speaking students indicates that computing vocabulary can represent a specific learning challenge. Terms such as instance, scope, runtime, exception, request, and endpoint require domain-specific understanding. General English proficiency does not necessarily imply familiarity with these technical meanings. The fourth level involves technical documentation and development tools. As students become more independent, they increasingly rely on: documentation, library references, error messages, issue trackers, technical forums, repository documentation, and development environments.

These resources are frequently written in English. This means that autonomy in programming may partly depend on the ability to interpret technical English. The fifth level concerns communication about code. Students must learn to explain solutions, ask questions, justify decisions, describe errors, and interpret feedback. These activities require not only programming knowledge but also the vocabulary needed to discuss programming. The literature therefore indicates that linguistic barriers extend beyond programming keywords. Simply translating terms such as while, if, or return would not address the broader issue. The challenge is distributed across the entire computing ecosystem. More recent research on multilingual prompting and artificial intelligence suggests that generative systems may reduce some linguistic barriers. Students can request explanations in their strongest language or translate technical documentation. However, linguistic accessibility does not necessarily guarantee technical correctness. Students still require enough computational knowledge to evaluate the explanation provided.

RQ2 Synthesis

The linguistic dimension includes: language of instruction, interpretation of task statements, technical vocabulary, access to documentation, interpretation of error messages, communication about code, and multilingual interaction with artificial intelligence systems. The results therefore suggest that language is part of access to computational knowledge, not merely a peripheral communication issue.

RQ3. Digital Competence and Technological Production

The reviewed literature strongly supports the distinction between digital access and advanced digital competence. Frameworks such as DigComp emphasize that digital competence includes much more than the ability to operate devices. Relevant components include: information search, source evaluation, digital content creation, problem solving, adaptation, security, and critical use.

This distinction has direct implications for technology education. Students may have extensive experience using technology while still having limited experience in technological production. For example, everyday digital use may involve: social media, messaging, streaming, online shopping, mobile applications, and entertainment.

Programming education requires additional competencies, including: organizing files, using development environments, interpreting technical messages, searching documentation, evaluating external solutions, testing, debugging, and creating computational artifacts.

The results therefore support the conceptual distinction between digital consumption and technological production. This distinction becomes even more relevant in the context of generative artificial intelligence. Artificial intelligence systems can reduce the technical difficulty of producing a first version of a program. However, students still need to

evaluate: whether the solution is correct, whether it satisfies requirements, whether edge cases are covered, whether the code is secure, and whether it can be adapted.

Digital competence in programming education therefore increasingly includes evaluative and critical capabilities.

RQ3 Synthesis

The dimensions of digital competence most relevant to technology education include: digital autonomy, information search, information evaluation, problem solving, tool adaptation, digital creation, and technological production. Access and frequent use alone do not provide sufficient evidence of these competencies.

RQ4. Pedagogical Strategies for Reducing Learning Barriers

The literature identifies several pedagogical strategies that may mediate learning difficulties. The most recurrent strategies include: diagnosis, scaffolding, visualization, formative feedback, debugging activities, active learning, pair programming, leveling activities, and differentiated support.

These strategies address different aspects of the learning process. Diagnosis can help identify differences in prior experience before those differences accumulate throughout a course. Scaffolding provides temporary structures that allow students to perform tasks they cannot yet complete independently. Visualization makes invisible computational processes more observable. Feedback can redirect attention toward the reasoning process rather than simply providing a correct answer. Debugging activities develop systematic problem-solving strategies. Pair programming encourages students to verbalize their reasoning and compare approaches. Leveling activities may provide additional support to students who begin with less experience. An important pattern across these strategies is the relationship between support and autonomy. Support is most consistent with the model proposed in this study when it is temporary and gradually reduced. The objective is not to keep students dependent on instructional assistance but to help them develop the ability to solve increasingly complex problems independently.

Artificial intelligence introduces an emerging form of scaffolding. Generative systems can provide explanations, examples, translations, and feedback. However, their educational value depends heavily on how they are used. A system that directly generates a complete answer may reduce the need for student reasoning. A system that provides questions, explanations, or test cases may preserve more of the problem-solving process (OECD, 2026).

RQ4 Synthesis

Pedagogical mediation appears most consistent with effective support when it: identifies student needs early, provides targeted assistance, makes reasoning visible, encourages active problem solving, uses feedback diagnostically, and gradually reduces external support.

RQ5. Relevance to the Latin American Context

The literature and institutional documents reviewed show that Latin America has expanded initiatives related to: digital competence, programming, coding, computational thinking, and Computer Science education. However, implementation is not uniform across the region. Countries differ in: infrastructure, connectivity, teacher preparation, curriculum design, institutional capacity, and access to technology education.

These differences create a regional context in which technological opportunities are unevenly distributed. The linguistic dimension also has particular relevance in Latin America. Portuguese and Spanish dominate formal education across much of the region. The international computing ecosystem, however, remains heavily dependent on English. As a result, students may experience multilingual demands even when their courses are officially monolingual. A Brazilian student may receive instruction in Portuguese while using: English programming keywords, English error messages, English documentation, English repositories, and English technical forums.

A similar situation occurs for Spanish-speaking students. This means that multilingualism may be embedded in technology learning even when it is not formally recognized as part of the curriculum. The review also identified a relative lack of research that simultaneously investigates: digital competence, programming education, multilingualism, and Latin America. This absence should be interpreted cautiously. It may represent a genuine research gap, but it may also reflect indexing practices, publication language, and the distribution of regional research across different databases.

RQ5 Synthesis

Three issues appear particularly relevant to Latin American technology education: persistent digital inequalities, uneven expansion of Computer Science education, and the interaction between Portuguese or Spanish educational environments and an English-dominated technical ecosystem (ECLAC, 2022; UNESCO, 2023b).

CROSS-DIMENSIONAL SYNTHESIS

Tables 2, 3, and 4 consolidate the evidence by analytical dimension, connect recurrent barriers to pedagogical responses, and relate each research question to its principal finding. The results show that learning barriers rarely operate in isolation. A student may experience difficulty with a programming task because of a conceptual

misunderstanding. Another may understand the computational concept but misunderstand the task statement. A third may understand both but have difficulty using the development environment or interpreting technical documentation.

Dimension	Main elements	Synthesis of evidence	Representative sources
Digital	autonomy, search, evaluation, production	everyday technology use does not necessarily imply technological production competence	DigComp; UNESCO; OECD; ECLAC
Cognitive	prior knowledge, abstraction, decomposition, problem solving, self-efficacy	different starting repertoires may influence introductory programming learning	Tang et al.; Tikva and Tambouris; Pires et al.
Linguistic	language, terminology, documentation, task interpretation	non-native English-speaking students may experience additional barriers	Guo; Soosai Raj et al.; Alaofi; Prather et al.
Educational	scaffolding, feedback, visualization, collaboration	educational strategies can mediate specific learning barriers	Rocha et al.; Bolloju et al.; pair programming research
Regional	digital access, curriculum, policy, language	Latin American contexts combine digital inequalities with multilingual technical environments	UNESCO; ECLAC

Table 2. Evidence by analytical dimension

Learning barrier	Possible manifestation	Pedagogical response
Limited digital experience	difficulty using development environments or files	guided introduction to tools
Limited prior knowledge	difficulty understanding basic programming concepts	diagnostic and leveling activities
Difficulty with abstraction	immediate attempt to code without planning	pseudocode and decomposition
Logical difficulty	inconsistent conditions or loops	tracing and visualization
Misconceptions	repeated conceptual errors	guided debugging
Low self-efficacy	withdrawal after repeated errors	progressive tasks and formative feedback
Linguistic difficulty	misunderstanding task requirements	clearer instructional language
Limited technical English	difficulty with documentation	bilingual technical glossary
Dependence on examples	difficulty transferring knowledge	progressive withdrawal of scaffolding
Dependence on AI	correct code without understanding	explanation, testing, and modification tasks

Table 3. Learning barriers and possible pedagogical responses

Research question	Main finding
RQ1	prior knowledge, problem solving, abstraction, self-efficacy, and mental models are recurring factors
RQ2	linguistic barriers extend beyond programming keywords to documentation, interpretation, and communication
RQ3	digital competence includes autonomy, critical evaluation, problem solving, and technological production
RQ4	progressive support, feedback, visualization, and active learning are relevant forms of educational mediation

Research question	Main finding
RQ5	Latin America combines uneven expansion of Computing Education, digital inequality, and an English-dominated technical ecosystem

Table 4. Relationship between research questions and main findings

Taken together, these findings support a multidimensional interpretation of learning barriers. The same observable difficulty may originate from different underlying conditions. A programming error may result from: task interpretation, prior knowledge, an inaccurate mental model, technical language, limited tool experience, and insufficient educational support.

The next section discusses these interactions and presents the Integrated Model of Barriers and Mediation in Technology Education.

DISCUSSION

The findings of this review indicate that learning difficulties in technology education are better understood when analyzed as the result of interactions among multiple learning conditions rather than as the consequence of a single factor. Prior knowledge, digital autonomy, linguistic repertoire, problem-solving ability, self-efficacy, and educational support appear across the literature as related components, even though they are often examined separately. The synthesis proposed in this study brings these lines of research together.

Technology Use Does Not Equal Technological Competence

One of the central arguments of this paper is that frequent technology use is not a sufficient indicator of competence for learning or producing technology.

DigComp 2.2 defines digital competence through a broad set of knowledge, skills, and attitudes that include information search and evaluation, communication, digital content creation, safety, and problem solving (Vuorikari, Kluzer and Punie, 2022). DigComp 3.0 maintains the general five-area structure while updating learning outcomes and integrating artificial intelligence, cybersecurity, misinformation, digital rights, and well-being across the framework (Cosgrove and Cachia, 2025). This evolution reinforces the argument that digital competence should not be confused with operational familiarity. Students may use smartphones, social networks, messaging applications, streaming platforms, and digital services intensively without having developed equivalent experience in: structured problem solving, technical information search, development environment configuration, interpretation of documentation, algorithm design, debugging, solution evaluation, and technological production. This distinction matters because technology education may implicitly assume that digitally active students need only learn the syntax of a programming language. The reviewed literature does not support this simplification. The analytical progression proposed in this paper,

Access → Use → Understanding → Problem Solving → Technological Production

highlights this distinction. The progression should not be interpreted as a validated linear scale. Students may demonstrate high competence in one component while having limited competence in another. Its conceptual purpose is to distinguish access and use from autonomy and production. This argument becomes even more relevant in the context of generative artificial intelligence. When an artificial intelligence system can produce code from a natural-language request, the ability to generate syntactically valid code becomes less informative as an isolated measure of technological competence. Other abilities become increasingly important: understanding generated code, evaluating whether it satisfies requirements, identifying limitations, designing tests, detecting vulnerabilities, comparing alternative solutions, and modifying code when requirements change. Generative artificial intelligence may therefore reduce operational barriers without eliminating conceptual barriers.

Student Heterogeneity as a Structural Condition

Another important finding is that student heterogeneity should be considered a normal condition of introductory technology education rather than an exception. Introductory courses may include students with very different educational and technological histories. Some students have previous programming experience. Others have strong experience using digital tools but no programming experience. Some have a strong mathematical background. Others have experience solving practical technical problems. Some are comfortable using computers but unfamiliar with development environments. This diversity is likely to increase as Computing Education reaches broader populations. A curriculum designed around an implicit assumption of prior technological familiarity may therefore transform initial differences into cumulative advantages. Students who are already familiar with core concepts may understand early content more quickly. They may then complete more exercises and gain additional experience. Students who must learn tools, terminology, logical representations, and programming concepts simultaneously may face a different learning burden. This review does not quantitatively demonstrate a cumulative effect. However, the

integration of the reviewed literature supports it as a plausible conceptual hypothesis. For this reason, initial diagnosis should be understood as a mechanism for educational support rather than student selection. The relevant question should not be: “Who is ready to learn programming?” It should instead be: “What forms of support are needed for students with different starting points to progress toward the same learning objectives?”

Code as the Visible Stage of a Larger Process

One of the most important implications of the cognitive dimension is that code represents only one stage of problem solving. This paper proposed the following process: Problem Statement → Interpretation → Decomposition → Abstraction → Algorithm → Implementation → Execution → Debugging → Revision

This representation has important consequences for both assessment and feedback. When a program produces an incorrect result, the problem may not have originated during coding. A student may have: misunderstood the task statement, incorrectly identified inputs or outputs, selected an inappropriate strategy, created an incorrect algorithm, created a correct algorithm but implemented it incorrectly, and failed to consider important test cases.

These situations may produce similar visible results but require different instructional responses. Correction based only on the final program may therefore provide insufficient information about the learning process. Diagnostic feedback should attempt to identify the point at which the difficulty emerged. Questions such as:

- What problem are you trying to solve?
- What information do you receive as input?
- What should the program produce?
- How would you solve the problem without writing code?
- Which part of the solution needs to repeat?
- How do you know when the repetition should stop?

may reveal difficulties that are not visible through syntax inspection alone.

Mathematics and Logic Without Determinism

The review identified research associating mathematical background, prior knowledge, and introductory programming outcomes. However, interpreting this literature as evidence that students with stronger mathematics necessarily learn programming more effectively would be inappropriate. Programming and mathematics share certain forms of abstraction, representation, and structured reasoning. Programming also includes domain-specific competencies such as: program state tracing, temporal execution, debugging, software tool use, error interpretation, and interaction with development environments.

Mathematical ability may therefore form part of a useful prior repertoire, but it does not provide a complete explanation of programming learning. This interpretation matters pedagogically. When students struggle with a conditional structure, repeating the syntax of if may not address the actual difficulty. The educator may first need to determine whether the learner understands: comparison, Boolean values, conditions, and the consequences of a decision. Technology education should therefore distinguish the computational concept from its syntactic representation.

Language as Part of Access to Computational Knowledge

The literature reviewed in this study provides consistent evidence that language can influence access to programming knowledge. Guo (2018) examined barriers faced by non-native English speakers learning programming. Soosai Raj, Ketsuriyonk, Patel and Halverson (2018) investigated the role of native language in programming learning. Later studies focused on computing terminology, English-medium introductory programming, and multilingual Computing Education.

More recent work extends this discussion to generative artificial intelligence and multilingual interaction. These studies support an interpretation in which linguistic barriers extend far beyond programming keywords such as if, while, or return. The barrier may occur in: explanations, task statements, technical terminology, error messages, documentation, forums, feedback, collaboration, and interaction with artificial intelligence systems.

English therefore functions partly as a gateway to computational information. Consider the error message: `TypeError: cannot read properties of undefined`. To use this message as diagnostic information, a learner may need to perform several operations. First, the student must understand the message linguistically. Second, the student must understand the computing concepts involved. Third, the message must be connected to the current state of the program. Finally, the student must formulate and test a hypothesis. A single technical message can therefore mobilize digital, cognitive, and linguistic competencies simultaneously.

Technical-Computing Linguistic Competence

The synthesis also supports a distinction between general language proficiency and domain-specific technical-computing linguistic competence. General language proficiency concerns a person's ability to use a language across

broad contexts. Technical-computing linguistic competence concerns the ability to understand and use specialized terminology in computational contexts. Examples include: instance, scope, runtime, dependency, exception, thread, request, response, payload, and endpoint. Several of these terms have ordinary meanings in English but specialized meanings in computing. Explicit teaching of this vocabulary should not be interpreted as reducing academic rigor. It can instead provide a bridge toward professional terminology. In Portuguese- and Spanish-speaking contexts, bilingual glossaries may support this transition.

The objective is not to permanently replace English technical terminology, but to connect terminology with concepts in ways that students can understand and later use independently.

Latin American Courses as Functionally Multilingual Environments

The linguistic dimension becomes particularly relevant in Latin America. A course may be taught entirely in Portuguese or Spanish while students continue to use: English programming keywords, English error messages, English documentation, English repository descriptions, English library references, and English technical forums.

A formally monolingual course may therefore function as a multilingual learning environment. This observation broadens the meaning of multilingualism in Computing Education. Multilingualism does not necessarily require students to be enrolled in an officially bilingual educational program. It may be embedded in the technological environment itself. This also means that language becomes part of technological autonomy (Jiang, 2026).

Students who can interpret documentation, search technical information, and communicate about code across linguistic boundaries may have access to a wider range of resources for independent problem solving.

Educational Mediation as the Main Area of Intervention

The review led to a refinement of the initial conceptual model. Digital, cognitive, and linguistic dimensions are not equivalent in function to the educational dimension. The first three primarily represent learner- and context-related conditions. The educational dimension represents the area where direct intervention is most feasible. Educational institutions cannot immediately change a student's educational history. They cannot instantly eliminate digital inequality. They cannot immediately change a learner's linguistic repertoire. They can, however, change: instructional materials, task sequencing, scaffolding, feedback, collaboration, visualization, assessment, and opportunities for practice.

The educational dimension therefore functions primarily as mediation. Educational design can reduce some barriers. It can also amplify them when poorly aligned with student needs.

Figure 4 presents the Integrated Model of Barriers and Mediation in Technology Education. The model organizes the relationship as follows: input conditions include digital, cognitive, and linguistic dimensions. These conditions may interact and create learning barriers. Educational mediation includes: diagnosis and assessment, scaffolding and feedback, visualization and modeling, collaboration and active learning, linguistic support, and critical use of artificial intelligence. The intended outcome is progressive learning leading toward: autonomy and technological production. The model does not assign statistical weights to the dimensions and does not claim deterministic causal relationships. Its purpose is to organize patterns identified across the reviewed literature.

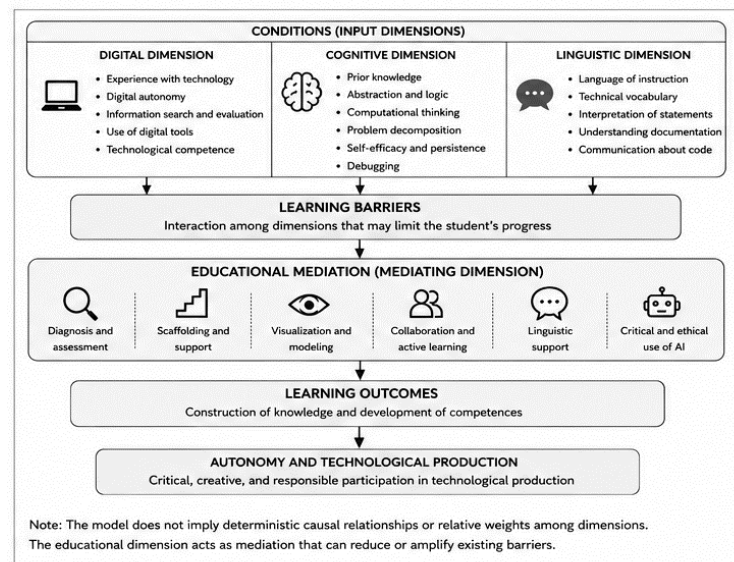


Figure 4. Integrated Model of Barriers and Mediation in Technology Education.

A Potential Cumulative Effect of Learning Barriers

The integration of the dimensions allows an additional conceptual hypothesis to be proposed. Learning barriers may accumulate. Consider a student with: limited experience using desktop computers, no prior programming experience, limited structured problem-solving experience, and limited technical English vocabulary.

During a single programming activity, this student may simultaneously need to learn: how to use the development environment, how to interpret technical terminology, how to understand the task statement, how to structure the solution, how to write the syntax, and how to interpret errors.

Another student may encounter the same activity while already possessing experience in several of these areas. Both students receive the same formal instruction. However, the cognitive and operational demands placed on them differ. The present study does not empirically demonstrate a statistical cumulative effect. This proposition should therefore be interpreted as a conceptual hypothesis emerging from the synthesis. Empirical validation of this interaction represents an important direction for future research.

Generative Artificial Intelligence as Barrier Reduction and Barrier Reconfiguration

Generative artificial intelligence is particularly relevant to the integrated model because it may operate across multiple dimensions. Within the linguistic dimension, artificial intelligence can: translate, explain terminology, simplify technical documentation, and allow interaction in a learner's preferred language.

Within the digital dimension, it can: help search for solutions, generate code, propose tests, and explain development tools. Within the cognitive dimension, it can: decompose problems, generate examples, explain algorithms, and suggest strategies.

About the educational dimension, it can function as a source of feedback and scaffolding. However, this support creates a new challenge. As artificial intelligence becomes more capable of producing complete answers, the ability to critically evaluate those answers becomes increasingly important. Students with stronger prior knowledge may use artificial intelligence to: compare alternatives, generate tests, verify hypotheses, identify weaknesses, and improve code.

Students with weaker conceptual foundations may use the same technology mainly to obtain completed solutions. The same tool may therefore produce different educational outcomes depending on the learner's existing competence. This raises an important question for digital inequality. Artificial intelligence may democratize access to sophisticated technological outputs while simultaneously increasing the value of competencies required to evaluate those outputs.

Implications for Student Assessment

The integrated model also has consequences for assessment. Traditional programming assessment often focuses on the final program. In an environment where generative systems can produce functional code, the final artifact provides increasingly limited evidence of student understanding. Assessment may need to place greater emphasis on: explanation, modification, debugging, testing, comparison, justification, and critical evaluation.

For example, students may receive a functioning program and be required to:

1. explain how it works;
2. identify limitations;
3. create additional test cases;
4. modify a requirement;
5. adapt the program;
6. justify the changes made.

Another activity may provide AI-generated code containing subtle errors and ask students to identify and correct them. These approaches shift assessment from code production alone toward computational understanding.

Contribution of the Integrated Model

The primary conceptual contribution of this review is the integration of research areas that are frequently examined separately. Digital competence research focuses on access, autonomy, problem solving, and digital creation. Computing Education research examines prior knowledge, computational thinking, misconceptions, debugging, and pedagogy. Multilingual education research examines language as a condition for access to knowledge. The integrated model connects these perspectives.

It proposes that digital, cognitive, and linguistic conditions can interact to shape learning barriers, while educational mediation can reduce, compensate for, or in some cases amplify those barriers. This model offers a framework for thinking about technology education without reducing learning difficulties to statements such as:

"The student lacks logic."

"The student is not good at mathematics."

"The student does not know English."

or

"The student does not know how to use technology."

Each of these statements may describe one part of a more complex situation. Understanding that complexity provides a stronger basis for designing educational interventions.

PRACTICAL IMPLICATIONS

The findings of this review suggest that institutions and educators responsible for programming and technology education should treat student heterogeneity as an expected condition of the learning process. Differences in prior knowledge, technological autonomy, linguistic repertoire, and problem-solving experience may create different learning needs even when students receive the same content, resources, and instructional time. A central practical implication is therefore that equal access to instruction should not be confused with equal learning conditions. Providing the same programming language, digital platform, learning environment, and assessment to all students is important, but it may not be sufficient when students begin from substantially different starting points. The integrated model proposed in this study suggests interventions at four levels: diagnosis, instructional design, learning support, and progressive development of autonomy.

Diagnose Before Assuming

Initial diagnostic activities can help educators identify differences in: previous programming experience, familiarity with development environments, logical reasoning, structured problem solving, technical terminology, digital autonomy, and self-efficacy. These activities should not be used to exclude students or make deterministic predictions about their potential. Their purpose is to identify support needs.

A diagnostic activity may, for example, ask students to interpret a problem, describe a solution without code, identify inputs and outputs, and explain how they would search for additional technical information. This type of activity can reveal competencies that are not visible in a syntax-based test. The results may then guide leveling materials, additional practice, mentoring, differentiated scaffolding, or extension activities.

Develop Digital Autonomy Progressively

Introductory programming courses should avoid assuming that all students are equally familiar with the technical environments required for programming. Before or during initial programming activities, some students may need support with: file organization, development environments, command-line interfaces, execution of programs, technical documentation, version control, and interpretation of error messages. These competencies do not necessarily require separate courses. They can be integrated into programming activities. For example, an exercise on functions may also include guided consultation of official documentation. A debugging activity may simultaneously teach interpretation of error messages and technical search strategies. This integration supports the gradual development of technological autonomy.

Teach the Problem Before the Code

The review suggests that students may benefit when problem representation is separated from implementation. Before writing code, learners can be asked to identify:

- What information is provided?
- What output is required?
- Which rules or restrictions apply?
- Which operations must repeat?
- Which decisions are required?
- Which edge cases may exist?

Students may then represent the solution using pseudocode, flowcharts, examples, or structured natural language before moving to code. This approach makes it easier to identify whether a difficulty is related to problem understanding or programming syntax.

Make Invisible Processes Visible

Computational processes are often difficult for beginners because program state is not directly visible. Educators can use: tracing tables, flowcharts, memory diagrams, execution traces, visual representations of data structures, and step-by-step debugging. These representations can help students develop mental models of program behavior. They

are especially useful for concepts such as: loops, functions, arrays, recursion, object-oriented programming, and state changes. The goal is not to make visual support permanent. It should be gradually reduced as students develop stronger internal representations.

Use Feedback to Support Investigation

Feedback should focus not only on whether a solution is correct but also on the reasoning that produced it. For example, instead of writing:

"Change < to <=."

an educator might ask:

"What is the final value that should be processed? Does your current condition allow that value to be reached?"

The second form keeps the student involved in diagnosis. Similarly, rather than correcting an entire algorithm, the educator can ask students to explain: expected behavior, current behavior, possible causes, and evidence supporting their hypothesis. This approach supports the development of debugging as a transferable competence.

Treat Debugging as a Learning Objective

Debugging should be taught as a normal programming activity rather than as a secondary task that occurs only after failure. An instructional sequence may provide code containing different categories of problems: syntax errors, logical errors, incorrect conditions, infinite loops, incorrect assumptions, and unhandled edge cases.

Students can then be asked to:

1. reproduce the problem;
2. classify the error;
3. formulate a hypothesis;
4. test that hypothesis;
5. correct the code;
6. explain the underlying cause.

This approach develops problem-solving skills that are not measured when students only reproduce correct examples.

Teach Technical-Computing Terminology Explicitly

Technical terminology should be treated as part of the curriculum, particularly in multilingual contexts. Table 5 presents a compact glossary in which terms are connected to contextual explanations. A bilingual or multilingual version can add translations while preserving the international terminology students will encounter in professional documentation.

Term	Contextual explanation
loop	a structure used to repeat instructions
runtime	the period during which a program is executing
exception	an event that interrupts or changes normal program flow
scope	the region in which an identifier can be accessed
instance	a concrete object created from a class
endpoint	an access point to a resource or operation in a service
request	a message sent to a system asking for an operation or resource
response	the information returned by a system after a request

Table 5. Technical-computing terminology glossary.

Reduce Linguistic Complexity When It Is Not the Learning Objective

Programming tasks should be cognitively appropriate without introducing unnecessary linguistic complexity. A long or ambiguous task statement may transform a programming exercise into an unintended reading comprehension assessment. This does not mean that all tasks should be simplified. It means that linguistic complexity should be intentional.

If the learning objective is a conditional structure, the main challenge should be the computational decision rather than an unnecessarily complex statement. As students progress, more open and linguistically complex problems can be introduced.

Structure Collaborative Learning

Pair programming and other collaborative strategies may help students verbalize reasoning and compare solutions. However, collaboration should be structured. The driver can control the keyboard and implementation. The navigator can monitor logic, question decisions, and identify possible issues. Roles should rotate. This helps reduce the risk that more experienced students dominate the activity while less experienced students become passive observers. Collaborative activities may also support multilingual students by allowing concepts to be negotiated through different linguistic resources.

Use Generative AI as Support, Not Replacement

Generative AI can support learning when its role is explicitly defined. Potential uses include: explaining error messages, translating documentation, generating test cases, comparing solutions, simplifying technical explanations, producing additional examples, and asking guiding questions.

The educational value depends on the level of assistance requested. Prompts such as:

- "Explain this error without changing my code."
- "Ask me three questions that may help me locate the problem."
- "Generate five test cases without writing the implementation."
- "Compare these two approaches and identify strengths and weaknesses."

preserve a greater degree of student reasoning than:

- "Solve this problem for me."

Institutions should therefore focus on the quality of AI-supported learning activities rather than treating AI use only as a binary issue of permission or prohibition.

Reconsider Programming Assessment

The increasing availability of code-generating systems reduces the value of assessments based only on the final source code. Assessment can instead include: oral explanation, code modification, debugging, test design, comparison of alternatives, code review, analysis of AI-generated code, and justification of design decisions. For example, students may receive a functioning program and be asked to:

1. explain its logic;
2. identify possible weaknesses;
3. create additional tests;
4. modify a requirement;
5. update the program;
6. justify the changes.

This type of assessment provides stronger evidence of understanding.

POLICY AND COMMUNICATION IMPLICATIONS

For institutions and policymakers, reducing learning barriers requires coordinated action across curriculum, teacher development, infrastructure, language support, assessment, and governance. The synthesis suggests that these are not only educational issues. They also affect access to technical information, participation in digital society, communication equity, and the capacity of individuals and institutions to engage critically with emerging technologies. Policies should therefore distinguish basic access from digital autonomy and technological production and should connect digital inclusion with meaningful participation rather than device availability alone (ECLAC, 2022; OECD, 2023, 2026; UNESCO, 2023a, 2023b).

Regional Policy Priorities

Four priorities follow from the synthesis: (1) pair investments in connectivity and equipment with digital-skills development and opportunities for technological production; (2) expand multilingual access to technical resources in Portuguese and Spanish, while addressing underrepresented linguistic communities and preserving pathways to international technical terminology; (3) establish clear educational AI-governance guidance covering critical AI literacy, transparency, human oversight, assessment, and evaluation of generated outputs; and (4) strengthen institutional capacity through educator development, diagnostic support, accessible learning resources, and monitoring of whether policies reduce or reproduce digital and linguistic inequalities.

LIMITATIONS

The findings of this study should be interpreted in light of several limitations. First, this is an integrative literature review rather than a meta-analysis. The reviewed publications use different research designs, populations, theoretical frameworks, and measurement instruments. It is therefore not possible to compare effect sizes across the digital, cognitive, linguistic, and educational dimensions. Second, computational thinking and digital competence do not have a single universal operational definition. Different studies measure these concepts using tests, self-reports, performance tasks, observational instruments, or institutional frameworks. This diversity limits direct comparison. Third, the review includes studies conducted across different educational levels. Research conducted with K-12 students cannot automatically be generalized to technical or higher education. For this reason, the conclusions are primarily conceptual and pedagogical rather than predictive.

Fourth, although the search strategy included Portuguese and Spanish, the international literature identified in this review remains strongly concentrated in English.

This limitation is particularly relevant because the study itself focuses on linguistic barriers. Fifth, the literature integrating programming education, multilingualism, digital competence, and Latin America remains comparatively limited. Regional interpretations therefore combine international evidence with Latin American institutional and policy documents. They should not be interpreted as evidence that all identified barriers occur in the same way across all Latin American countries.

Sixth, generative artificial intelligence is developing rapidly. Educational practices, system capabilities, and institutional policies may change more quickly than other areas reviewed in this study, so conclusions related to AI may require frequent updating. Seventh, the absence of a complete auditable search and screening log prevents replication of exact retrieval counts and limits the review's reproducibility. Finally, the Integrated Model of Barriers and Mediation proposed in this paper has not yet been empirically validated. It is a conceptual synthesis and does not establish causal relationships, relative weights, or predictive accuracy.

FUTURE RESEARCH AGENDA

The limitations and conceptual synthesis identify several directions for future research:

Empirical Validation of the Integrated Model

Future studies can operationalize the proposed dimensions and examine their relationships with student performance and learning experiences. Potential variables include: digital autonomy, prior programming knowledge, problem-solving ability, technical English competence, self-efficacy, and perceived educational support.

Quantitative studies could examine the relative contribution of these variables. Qualitative research could investigate how students experience the interaction among these barriers. Mixed-method studies may be particularly useful.

Longitudinal Research

Many studies examine students at a single point in time. Longitudinal research could investigate whether initial differences remain stable or decrease over the duration of a course. Questions include:

Do students with limited prior programming experience remain at a disadvantage after one semester?

Which forms of scaffolding are associated with reduced differences over time?

Which competencies develop most strongly during introductory programming courses?

These questions would help distinguish temporary starting differences from persistent learning barriers.

Portuguese, Spanish, and English in Programming Education

Latin America provides a particularly relevant environment for multilingual Computing Education research. Future studies could compare how students use Portuguese, Spanish, and English when: reading documentation, interpreting errors, searching for solutions, discussing code, using AI assistants, and learning technical terminology.

The objective should not be to determine which language is superior. The objective should be to understand how students navigate between different linguistic resources while solving computational problems.

Technical-Computing Linguistic Competence

The concept of technical-computing linguistic competence proposed in this paper should be examined empirically. An assessment instrument could evaluate students' understanding of terms such as: instance, scope, runtime, exception, dependency, endpoint, request, and response. The results could then be compared with: documentation comprehension, debugging performance, programming performance, and technological autonomy.

Such research could clarify whether technical terminology represents a distinct educational dimension.

Cumulative Interaction of Barriers

The integrated model suggests that barriers may interact. Future studies could test whether the simultaneous presence of several barriers produces greater difficulty than each barrier independently. For example: Does limited digital

autonomy combined with limited technical English create greater difficulty in programming than either condition alone? Does low prior knowledge become more significant when students also have limited access to educational support? These questions require empirical testing.

Generative AI and Digital Inequality

Another important research direction concerns whether generative AI reduces or increases competence gaps. More experienced students may use AI to: compare solutions, generate tests, review code, refactor, and verify hypotheses. Less experienced students may use the same systems primarily to obtain completed answers. Future studies could compare: prompt quality, ability to identify AI-generated errors, knowledge retention, problem-solving transfer, and subsequent performance without AI assistance. This research could clarify whether AI acts as an equalizing technology or whether it creates new forms of digital inequality.

Authentic Assessment in AI-Rich Environments

Research is also needed on assessment approaches that remain meaningful when code generation is widely available. Possible formats include: debugging, code review, oral explanation, program modification, test design, solution comparison, and critical evaluation of generated code. Future research should examine which formats provide the strongest evidence of conceptual understanding and autonomous problem solving.

Linguistically Underrepresented Populations

Latin America contains substantial linguistic diversity beyond Portuguese and Spanish. Indigenous languages and other linguistic communities remain underrepresented in the literature identified in this review. Future Computing Education research should investigate how these populations access programming education, technical terminology, and digital learning resources. The relative absence of such research represents an important inclusion gap.

CONCLUSION

The expansion of technology education has increased student access to programming, computational thinking, and digital production. However, increased access does not mean that all learners begin with the same conditions for transforming technology into learning and technological production.

This integrative review shows that barriers to technology learning are multidimensional. The digital dimension includes autonomy, use of tools, information search and evaluation, digital problem solving, and technological production. The cognitive dimension includes prior knowledge, abstraction, decomposition, computational thinking, self-efficacy, mental models, and debugging. The linguistic dimension includes language of instruction, interpretation of task statements, technical terminology, documentation, error messages, and communication about code. These factors may interact. As a result, similar observable difficulties may have different underlying causes.

Incorrect code may result from a syntactic problem, but it may also reflect a misunderstanding of the problem, an inaccurate mental model, a linguistic barrier, limited tool experience, or insufficient instructional support.

Based on this synthesis, the paper proposes an Integrated Model of Barriers and Mediation in Technology Education. The model organizes digital, cognitive, and linguistic conditions as potential sources of learning barriers and positions the educational dimension primarily as a space of mediation. This shifts part of the discussion away from a question focused exclusively on the learner: “What does the student lack?” toward a broader educational question: “What conditions can be created to support students with different starting points in developing technological autonomy?”. This perspective is particularly relevant in multilingual contexts. Latin American students may receive instruction in Portuguese or Spanish while simultaneously interacting with English programming keywords, documentation, error messages, libraries, repositories, and professional communities. Access to computational knowledge therefore also has a linguistic dimension. Generative artificial intelligence creates an additional transformation. When systems can produce functional code automatically, the existence of working code becomes an increasingly weak indicator of learning when considered alone. Understanding, testing, evaluating, modifying, and justifying technological solutions become more important. Technology education must therefore balance support with autonomy. Diagnosis, scaffolding, visualization, formative feedback, debugging, collaboration, linguistic support, and critical use of artificial intelligence can help students develop the required competencies without reducing academic expectations. The main conclusion of this review is that digital inclusion in technology education does not end with access to computers, connectivity, platforms, or artificial intelligence systems. It also requires conditions that allow students to understand problems, interpret technical language, formulate strategies, evaluate solutions, and progressively participate in technological production. This broader understanding of inclusion is particularly important as Computer Science education expands to increasingly diverse student populations.

These findings also carry direct policy implications for Latin America. Digital inclusion should not be reduced to connectivity or device provision. Regional and institutional policies should combine infrastructure with digital and AI literacy, multilingual access to technical information, educator preparation, institutional capacity, and responsible AI governance. From a communication-policy perspective, these conditions shape who can access, interpret, evaluate, and produce digital information and who can participate meaningfully in increasingly AI-mediated

educational and social environments. Technology education is therefore part of a broader regional agenda of digital inclusion, communication equity, and institutional capacity (ECLAC, 2022; OECD, 2023, 2026; UNESCO, 2023a, 2023b).

REFERENCES

1. Alaofi, S. and Russell, S. (2021) Computer terminology test for non-native English speaking students in CS1, in *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, New York, 1304. DOI: 10.1145/3408877.3439593.
2. Alaofi, S. and Russell, S. (2022) A validated computer terminology test for predicting non-native English-speaking CS1 students' academic performance, in *Proceedings of the 24th Australasian Computing Education Conference*, Association for Computing Machinery, 133-142. DOI: 10.1145/3511861.3511876.
3. Alaofi, S. and Russell, S. (2023) The use of English language to teach CS1 to non-native English speakers: Students perspective, in *Proceedings of the ACM Conference on Global Computing Education*, December 5-9, Hyderabad, India, Association for Computing Machinery, 15-21. DOI: 10.1145/3576882.3617931.
4. Bandura, A. (1997) *Self-Efficacy: The Exercise of Control*, W.H. Freeman, New York.
5. Bolloju, N., Malapati, A. and Upadhyay, P. (2026) The effects of starter template scaffoldings on coding anxiety in introductory programming courses, *ACM Transactions on Computing Education*, 26, 3, 1-30. DOI: 10.1145/3796506.
6. Cosgrove, J. and Cachia, R. (2025) *DigComp 3.0: European Digital Competence Framework – Fifth Edition*, Publications Office of the European Union, Luxembourg. DOI: 10.2760/0001149.
7. Economic Commission for Latin America and the Caribbean, ECLAC (2022) *Digital Agenda for Latin America and the Caribbean, eLAC2024*, United Nations, Santiago.
8. García, O. and Wei, L. (2014) *Translanguaging: Language, Bilingualism and Education*, Palgrave Macmillan, London.
9. Guo, P. J. (2018) Non-native English speakers learning computer programming: Barriers, desires, and design opportunities, in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, Association for Computing Machinery, New York, 1-14. DOI: 10.1145/3173574.3173970.
10. Jiang, J. (2026) Envisioning multilingual learning in Computer Science classrooms, in *Proceedings of the 2026 ACM Conference on International Computing Education Research*, Association for Computing Machinery, 51-52. DOI: 10.1145/3765965.3816675.
11. OECD (2023) *OECD Digital Education Outlook 2023: Towards an Effective Digital Education Ecosystem*, OECD Publishing, Paris. DOI: 10.1787/c74f03de-en.
12. OECD (2026) *OECD Digital Education Outlook 2026: Exploring Effective Uses of Generative AI in Education*, OECD Publishing, Paris. DOI: 10.1787/062a7394-en.
13. Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D. et al. (2021) The PRISMA 2020 statement: An updated guideline for reporting systematic reviews, *BMJ*, 372, n71. DOI: 10.1136/bmj.n71.
14. Pires, J., Gomes, A., Borges, A. R. and Correia, F. B. (2024) Predicting higher education student's aptitude to learn to program: A systematic literature review, in *Proceedings of the International Conference on Computer Systems and Technologies 2024*, Association for Computing Machinery, 242-248. DOI: 10.1145/3674912.3674949.
15. Prather, J., Reeves, B. N., Denny, P., Leinonen, J., MacNeil, S., Luxton-Reilly, A., Orvalho, J., Alipour, A., Alfageeh, A., Amarouche, T., Kimmel, B., Wright, J., Blake, M. and Barbre, G. (2025) Breaking the programming language barrier: Multilingual prompting to empower non-native English learners, in Seton, C. and Simon (Eds.) *Proceedings of the 27th Australasian Computing Education Conference*, Association for Computing Machinery, 74-84. DOI: 10.1145/3716640.3716649.
16. Rocha, H., Costa, E. and Tedesco, P. (2023) Helping to provide adaptive feedback to novice programmers: a framework to assist the teachers, in *Proceedings of the 18th Iberian Conference on Information Systems and Technologies (CISTI)*, IEEE, 1-6. DOI: 10.23919/CISTI58278.2023.10212000.
17. Sheard, J., Malmi, L., Szabo, C., Kinnunen, P., Tshukudu, E., Craig, M. and Mann, S. (2025) Pair programming as a pedagogical practice: Connections to Computing Education theories, in *Proceedings of the ACM Global Computing Education Conference 2025*, Association for Computing Machinery, 57-63. DOI: 10.1145/3736181.3747142.

18. Soosai Raj, A. G., Ketsuriyonk, K., Patel, J. M. and Halverson, R. (2018) Does native language play a role in learning a programming language?, in *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, New York, 417-422. DOI: 10.1145/3159450.3159531.
19. Tang, X., Yin, Y., Lin, Q., Hadad, R. and Zhai, X. (2020) Assessing computational thinking: A systematic review of empirical studies, *Computers & Education*, 148, 103798. DOI: 10.1016/j.compedu.2019.103798.
20. Tikva, C. and Tambouris, E. (2021) Mapping computational thinking through programming in K-12 education: A conceptual model based on a systematic literature review, *Computers & Education*, 162, 104083. DOI: 10.1016/j.compedu.2020.104083.
21. UNESCO Global Education Monitoring Report Team (2023a) Global Education Monitoring Report 2023: Technology in Education: A Tool on Whose Terms?, UNESCO, Paris.
22. UNESCO Global Education Monitoring Report Team and Sadosky Foundation (2023b) Computer Science as a Curriculum Subject in Latin America, background paper prepared for the Global Education Monitoring Report 2023, UNESCO.
23. Van Dijk, J. (2020) *The Digital Divide*, Polity Press, Cambridge.
24. Vuorikari, R., Kluzer, S. and Punie, Y. (2022) *DigComp 2.2: The Digital Competence Framework for Citizens - With New Examples of Knowledge, Skills and Attitudes*, EUR 31006 EN, Publications Office of the European Union, Luxembourg. ISBN 978-92-76-48882-8. DOI: 10.2760/115376.
25. Vygotsky, L. S. (1978) *Mind in Society: The Development of Higher Psychological Processes*, Harvard University Press, Cambridge, MA.
26. Wing, J. M. (2006) Computational thinking, *Communications of the ACM*, 49, 3, 33-35. DOI: 10.1145/1118178.1118215.